APCS 程式設計系列 · U08 · 實作中高級 · Python版

# 排序與二分搜尋 詳解

泡沫 / 選擇 / 插入排序 · 內建排序與自訂排序 · 穩定性 · $O(n^2)$ 與 $O(n \log n)$ · 二分搜尋 · 上下界 · 二分答案

## Part A 答案速查

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| B | A | C | D | A | B | C | A | B | D |
| **11** | **12** | **13** | | | | | | | |
| D | C | B | | | | | | | |

## Part A 程式識讀詳解

**第 1 題 答案：(B)**

- 這是泡沫排序的「一輪」：相鄰兩個比，前面大就交換。
- 5>2 換 → 2 5 8 1 9 3；5、8 不換；8>1 換 → 2 5 1 8 9 3；8、9 不換；9>3 換 → 2 5 1 8 3 9。

【錯】一輪只保證最大的 9 跑到最後，不是整個排好。

**第 2 題 答案：(A)**

- 選擇排序只跑 2 輪。第 1 輪：最小值 1 在第 3 格，和第 1 格的 4 交換 → 1 7 4 9 2 6。
- 第 2 輪：後面 7 4 9 2 6 的最小值 2，和第 2 格的 7 交換 → 1 2 4 9 7 6。

【錯】交換是「兩格對調」，7 會跑到原本 2 的位置，不是整段往後挪。

**第 3 題 答案：(C)**

- 插入排序只做到 i = 3：把前 4 個數排好，最後的 5 還沒處理。
- 3 插到 6 前面 → 3 6 8 2 5；8 不動；2 插到最前面 → 2 3 6 8 5。

**第 4 題 答案：(D)**

- 泡沫排序的交換次數 = 逆序對數（前面的數比後面大的對數）。
- 4 比 3、2、1 大：3 對；3 比 2、1 大：2 對；2 比 1 大：1 對；5 比 1 大：1 對。共 7 次。

【錯】選 10 是把所有比較次數 4 + 3 + 2 + 1 都算進去；只有真的交換才加 1。

**第 5 題 答案：(A)**

- l = 0、r = 9：m = 4，a[4] = 16 < 23，l = 5。
- l = 5、r = 9：m = 7，a[7] = 56 > 23，r = 6。
- l = 5、r = 6：m = 5，a[5] = 23，找到，break。

**第 6 題 答案：(B)**

- m = 4（16 < 20）→ l = 5；m = 7（56）→ r = 6；m = 5（23）→ r = 4。l > r 結束，共 3 圈。
- l = 5：剛好是「第一個 >= 20」的位置（a[5] = 23），也就是 lower_bound。

**第 7 題 答案：(C)**

- m = 4（5 > 3）→ r = 4；m = 2（3 <= 3）→ l = 3；m = 3（3 <= 3）→ l = 4。l = r = 4 結束。
- 條件是 <= x，所以找到的是「第一個 > 3」的位置，也就是 upper_bound。

【錯】把 <= 看成 <，就會選「第一個 >= 3」的 1。

**第 8 題 答案：(A)**

- key 是 (分數取負, 名字)：分數高的在前，同分再依名字的字典序。
- 92 的 Bob 第一；85 的 Amy、Eve 依字母 A < E；最後是 70 的 Dan。

【錯】只依分數排的話，穩定排序會保留原本順序 Eve、Amy，選成 B。

**第 9 題 答案：(B)**

- 只依長度排：fig 3、pear 4、kiwi 4、date 4、apple 5。
- Python 的排序是穩定的：長度都是 4 的 pear、kiwi、date 保持原本的先後。

【錯】以為同長度會再依字母排，就會選 A。

**第 10 題 答案：(D)**

- bisect_left(a, 3)：第一個 >= 3 的位置是 1。bisect_right(a, 3)：第一個 > 3 的位置是 4。
- 4 不在串列裡，bisect_left(a, 4) 是「4 應該插入的位置」，也就是 5 的位置 4。

**第 11 題 答案：(D)**

- 泡沫、選擇、插入排序都是 $O(n^2)$，約 $2\times10^{10}$ 次運算，一定超時。
- 內建排序是 O(n log n)，約 $3.6\times10^{6}$ 次，很快就完成。

**第 12 題 答案：(C)**

- m = (4 + 5) / 2 = 4（向下取整），ok(4) 成立，l = m 還是 4。
- l、r 都沒變，下一圈一模一樣：無窮迴圈。修正：m 改成 (l + r + 1) / 2，向上取整。

【錯】l = mid 一定要配向上取整的 mid。

**第 13 題 答案：(B)**

- 這是找「最小的 m，使 m × m >= 50」。m 依序是 25、13、7、10、9、8。
- 7 × 7 = 49 不夠，l = 8；之後 r 一路縮到 8。共 6 圈，答案 8。

# Part B 程式實作參考解答

## 實作 1 成績排名

### 解題思路

- 把每位同學存成（總分, 數學, 座號），用自訂規則排序一次就好。
- key 用 tuple：(-總分, -數學, 座號)，取負號就變成大到小。
- 範例 1 的前四位總分都是 240：數學 80 的 2、3、4 號依座號排，再來是數學 70 的 1 號。

【錯】只依總分排，同分的先後就亂了（C++ 的 sort 不穩定）；三個鍵都要寫進規則。

【錯】n 到 $2\times10^5$，不能用兩層迴圈的基本排序。

**Python 成績排名**

```python
import sys
data = sys.stdin.read().split()
n = int(data[0])
rows = []
for i in range(1, n + 1):
    c, e, m = map(int, data[3 * i - 2:3 * i + 1])
    rows.append((-(c + e + m), -m, i))
rows.sort()
print(*[r[2] for r in rows])
```

時間複雜度：O(n log n)

## 實作 2 泡沫排序追蹤

### 解題思路

- 照題目模擬：外層第 i 輪（從 0 開始數就是第 i + 1 輪），內層比到 n - 1 - i。
- 每次交換就把計數加 1；外層跑完第 k 輪時，把整個數列複製一份存起來。
- 相等的數不交換（前面「大於」後面才換），所以 3 3 不會互換。總交換次數 = 逆序對數。

【錯】存第 k 輪的結果要寫 snap = a[:] 複製一份；寫 snap = a 只是同一個串列多一個名字，最後會變成排好的樣子。

【錯】「第 k 輪」從 1 開始數，程式裡的 i 從 0 開始，判斷要寫 i + 1 == k。

**Python 泡沫排序追蹤**

```python
import sys
data = sys.stdin.read().split()
n, k = int(data[0]), int(data[1])
a = list(map(int, data[2:2 + n]))
swaps, snap = 0, None
for i in range(n - 1):              # 第 i + 1 輪
    for j in range(n - 1 - i):
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
            swaps += 1
    if i + 1 == k:
        snap = a[:]                 # 複製一份
print(*snap)
print(swaps)
```

時間複雜度：$O(n^2)$（$n \le 2000$，約 $2\times10^6$ 次比較）

### 實作 3 區間計數

**解題思路**

- 每次詢問都從頭數一遍是 O(nq) = $4\times10^{10}$，一定超時。
- 先排序一次；之後每個詢問的答案 =「第一個 > R 的位置」減「第一個 >= L 的位置」。
- 也就是 bisect_right(a, R) - bisect_left(a, L)，每次 O(log n)。

【錯】L 要用「第一個 >= L」、R 要用「第一個 > R」；兩邊都用同一種，等於 L 或 R 的數會被漏算或多算（範例的 [3, 7] 要算到兩個 3 和 7）。

【錯】輸出 $2\times10^5$ 行，先收集起來最後一次 print，不要每個詢問都 print。

**Python 區間計數**

```python
import sys
from bisect import bisect_left, bisect_right
data = sys.stdin.read().split()
n, q = int(data[0]), int(data[1])
a = sorted(map(int, data[2:2 + n]))
out = []
idx = 2 + n
for _ in range(q):
    L, R = int(data[idx]), int(data[idx + 1])
    idx += 2
    out.append(bisect_right(a, R) - bisect_left(a, L))
print("\n".join(map(str, out)))
```

時間複雜度：O((n + q) log n)

### 實作 4 貨運排班

**解題思路**

- C 越大，需要的卡車越少：答案有單調性，可以二分答案。
- ok(C)：從頭貪心裝車，裝得下就裝，裝不下就換下一台，最後看用了幾台 <= m。O(n)。
- C 至少是最重的那一箱（不然那箱永遠裝不上），最多是全部的總和：在這個範圍找「第一個 ok 的 C」。
- 範例 1：C = 18 時分成 7 2 5 和 10 8 兩台；C = 17 要三台。

【錯】l 從 0 或 1 開始的話，C 比某一箱還輕，貪心會出錯（那一箱自己就超重）。下界要設成最重的一箱。

【錯】找「最小的成立值」：ok 就 r = mid，否則 l = mid + 1，mid 向下取整。

**Python 貨運排班**

```python
import sys
data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
w = list(map(int, data[2:2 + n]))
def ok(cap):                    # 上限 cap，m 台車夠嗎？
    cnt, cur = 1, 0
    for x in w:
        if cur + x > cap:
            cnt += 1
            cur = x
        else:
            cur += x
    return cnt <= m
l, r = max(w), sum(w)
while l < r:                    # 找第一個 ok 的 cap
    mid = (l + r) // 2
    if ok(mid):
        r = mid
    else:
        l = mid + 1
print(l)
```

時間複雜度：O(n log(總重))，約 $2\times10^5 \times 48$ 次

## 實作 5 夜市擺攤

### 解題思路

- 距離 d 越小越容易擺得下：答案有單調性，二分答案。
- ok(d)：座標排序後，第一攤放最左邊，之後只要和上一攤距離 >= d 就擺，最後看擺了幾攤 >= k。
- 範例 1：排序後 1 2 4 8 9，d = 3 可以擺 1、4、8；d = 4 只能擺 1、8 兩攤。

【錯】座標沒有排好，要先排序才能貪心。

【錯】找「最大的成立值」：ok 就 l = mid、否則 r = mid - 1，這時 mid 一定要向上取整 (l + r + 1) / 2，否則會無窮迴圈。

**Python 夜市擺攤**

```python
import sys
data = sys.stdin.read().split()
n, k = int(data[0]), int(data[1])
x = sorted(map(int, data[2:2 + n]))

def ok(d):                      # 間距至少 d，擺得下 k 攤嗎？
    cnt, last = 1, x[0]
    for v in x:
        if v - last >= d:
            cnt += 1
            last = v
    return cnt >= k

l, r = 1, x[-1] - x[0]
while l < r:                    # 找最後一個 ok 的 d
    mid = (l + r + 1) // 2      # 向上取整
    if ok(mid):
        l = mid
    else:
        r = mid - 1
print(l)
```

時間複雜度：O(n log n + n log(座標範圍))

## 自我診斷表

對完答案，數一數每個觀念錯幾題：全對圈綠、錯 1 題圈黃、錯 2 題以上圈紅。紅燈的觀念回去看講義那一段。

| 觀念 | 題號 | 錯幾題 | 燈號（圈一個） | 回去看哪裡 |
|---|---|---|---|---|
| **基本排序** | A1、A2、A3、A4、B2 | | 綠 黃 紅 | 講義第 1 段（某一輪之後、交換次數） |
| **自訂排序** | A8、A9、B1 | | 綠 黃 紅 | 講義第 2、3 段（多鍵、比較函式、穩定性） |
| **時間複雜度** | A11 | | 綠 黃 紅 | 講義第 4 段 |
| **二分搜尋** | A5 | | 綠 黃 紅 | 講義第 5 段（三大陷阱） |
| **上下界** | A6、A7、A10、B3 | | 綠 黃 紅 | 講義第 6 段 |
| **二分答案** | A12、A13、B4、B5 | | 綠 黃 紅 | 講義第 7 段 |

**覺得有幫助？幫老師一個忙**

- **訂閱** YouTube「莊老數學教室」：youtube.com/@chuanglaomath
- 看到老師的影片，順手**按個讚**
- 把這份免費講義**分享給同學**：wade-chuang.github.io/jenfoo/apcs（不用註冊，直接下載）